

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 1

**Duration :60 Mins
On Class
Total Marks:50**

Annexure A

TECHNICAL PRESENTATION

Code of Conduct:

*I _____ Karanya S (192421309) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

***Karanya S
192421309***

Annexure A

TECHNICAL PRESENTATION

Q1. Phases of a Compiler and Their Role

Understanding of Case

- a) Explains all compiler phases such as lexical, syntax, semantic, intermediate, optimization, and code generation.
- b) Describes the role of each phase in transforming source code into machine code.

Application of Concepts

- c) Applies compiler phases to detect errors like invalid tokens, missing syntax, and semantic mismatches.
- d) Demonstrates phase-wise error identification using real C program examples.

Analysis

- e) Analyzes how each phase depends on the previous phase for correct execution.
- f) Evaluates how early-stage errors (lexical/syntax) stop further compilation.

Solution Design

- g) Suggests integrating better error recovery techniques across phases.
- h) Proposes modular compiler design for flexibility and easier debugging.

Clarity

- i) Uses flowcharts to explain phase sequence clearly.
- j) Provides step-by-step examples showing how code is processed.

Q2. Lexical Analysis and Token Specification using LEX

Understanding of Case

- a) Defines tokens, lexemes, and patterns used in lexical analysis.
- b) Explains the role of the lexical analyzer in scanning source code.

Application of Concepts

- c) Demonstrates token generation using LEX tool with sample inputs.
- d) Applies regular expressions to identify identifiers, constants, and operators.

Analysis

- e) Analyzes common lexical errors such as invalid identifiers and illegal symbols.
- f) Evaluates how tokenization improves parsing efficiency.

Solution Design

- g) Designs token rules using regular expressions in LEX.
- h) Implements error-handling mechanisms in lexical analysis.

Clarity

- i) Provides clear LEX syntax examples for better understanding.
- j) Uses input-output illustrations for token generation.

Q3. Language Processors: Compiler vs Interpreter

Understanding of Case

- a) Differentiates between compiler and interpreter based on execution method.
- b) Explains how each processor converts high-level code.

Application of Concepts

- c) Applies concepts to languages like C (compiler) and Python (interpreter).
- d) Demonstrates how errors are handled differently in both.

Analysis

- e) Compares performance, speed, and memory usage of both processors.
- f) Evaluates advantages and disadvantages in real-world scenarios.

Solution Design

- g) Suggests using compilers for performance-critical applications.
- h) Recommends interpreters for rapid development and debugging.

Clarity

- i) Uses comparison tables for easy understanding.
- j) Includes real-life examples and execution flow diagrams.

Q4. Grouping of Compiler Phases and Optimization

Understanding of Case

- a) Explains grouping into front-end (analysis) and back-end (synthesis).
- b) Describes the role of each group in compilation.

Application of Concepts

- c) Applies grouping to simplify compiler design and debugging.
- d) Demonstrates how front-end handles errors and back-end improves performance.

Analysis

- e) Analyzes how grouping improves modularity and maintainability.
- f) Evaluates optimization techniques like constant folding and dead code removal.

Solution Design

- g) Designs efficient compiler structure using grouped phases.
- h) Suggests advanced optimization techniques for better performance.

Clarity

- i) Uses diagrams to represent front-end and back-end clearly.
- j) Provides before-and-after code examples for optimization.

Q5. Compiler Construction Tools and LEX Understanding of Concepts

- a) Explain compiler construction tools.
- b) Describe the structure of a LEX program.

Application of Concepts

- c) Apply LEX to generate tokens for a simple program.
- d) Demonstrate writing rules for identifiers and constants.

Analysis

- e) Analyze advantages of using LEX in compiler design.
- f) Evaluate limitations of LEX.

Solution / Design

- g) Design a simple lexical analyzer using LEX rules.

Clarity

- h) Show LEX workflow (input → scanner → tokens).

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-dept analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Compiler Construction Tools and LEX Understanding of Concepts

KARANYA S
192421309

Supervisor
Dr. Michael
Professor
Department of CSE

a) Compiler Construction Tools and LEX

- Compiler Construction Tools help in building a compiler efficiently.
- **Compiler Construction Tools**
 - LEX (Lexical Analyzer)
 - YACC (Syntax Analyzer)
 - Code Optimization
 - Code Generation
- **LEX**
 - Performs Lexical Analysis
 - Identifies Tokens
 - Removes Spaces and Comments
 - Sends Tokens to the Parser
- **Flow**
 - **Source Code → LEX → Parser (YACC) → Intermediate Code → Machine Code**

b) Explain the role of LEX

- Scans the source program character by character.
- Identifies tokens such as keywords, identifiers, and operators.
- Removes whitespace and comments.
- Detects invalid lexical patterns.
- Passes tokens to the parser (YACC) for syntax analysis.

c) Explain Compiler Construction Tools and LEX Structure

- **1. Compiler Construction Tools**

- LEX (Lexical Analyzer)
- YACC (Syntax Analyzer)
- Code Optimization
- Code Generation
- **Output:** Intermediate Representation (IR)

- **2. LEX (Lexical Analyzer)**

- Reads Source Code
- Identifies Tokens
- Removes Spaces and Comments
- Sends Tokens to Parser
- **Output:** Token Stream to Parser (YACC)

d) Explain the role of Compiler Construction Tools and LEX

- **Source Code**
- → **LEX** (identifies tokens)
- → **Parser (YACC)** (checks syntax)
- → **Intermediate Code**
- → **Code Generator** (generates machine code)

e) Analyse advantages of Compiler Construction Tools and LEX

- Reduces compiler development time.
- Automatically generates lexical analyzers.
- Improves accuracy in token recognition.
- Easy to modify and maintain lexical rules.
- Integrates efficiently with parser generators (YACC).

f) Importance of Compiler Construction Tools and LEX

- Simplifies compiler development.
- Automates lexical analysis.
- Reduces coding effort and errors.
- Improves compiler efficiency.
- Makes the compiler easier to maintain and update.

g) Design of Compiler Construction Tools and LEX

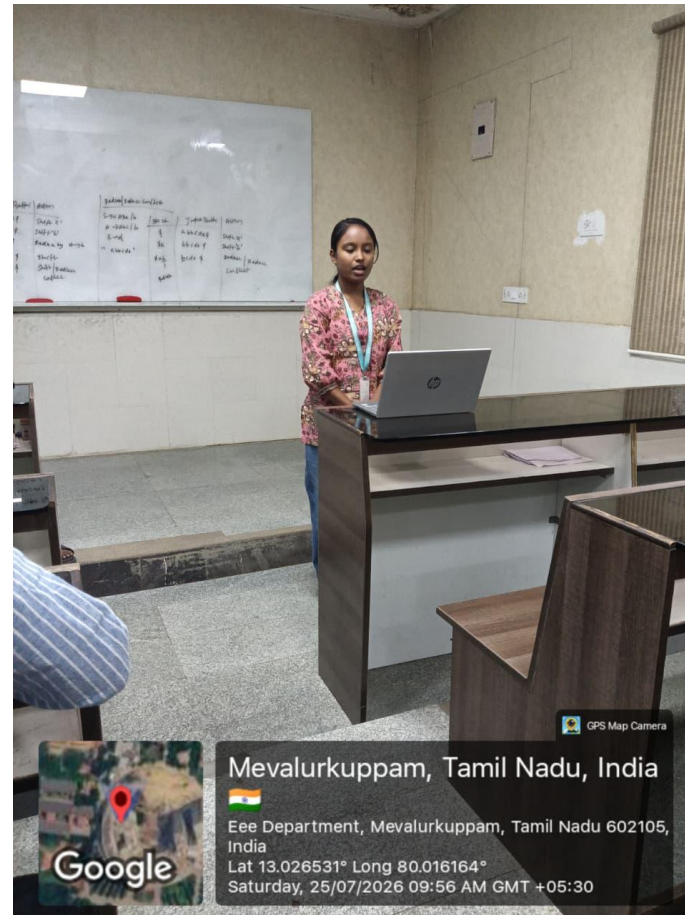
Steps:

- Write lexical rules in **LEX**.
- Generate the lexical analyzer.
- Scan source code and identify tokens.
- Pass tokens to the parser (**YACC**).
- Generate target machine code.

h) Example Flow

- **Input:**
- `int a = b + 5;`
- **Processing:**
- **LEX** → Identifies tokens (int, a, =, b, +, 5)
- **Parser (YACC)** → Checks syntax
- **Code Generator** → Produces machine code
- **Output:**
Executable machine code

i) Geotag



THANKYOU